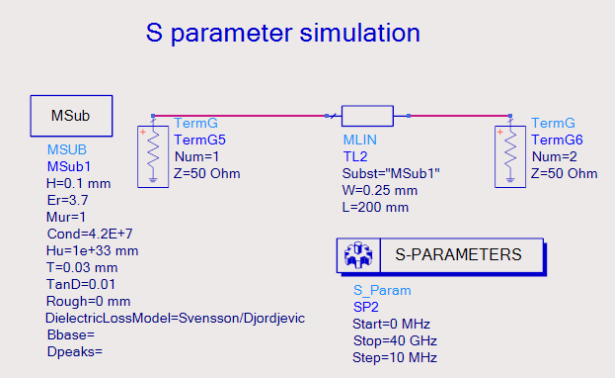
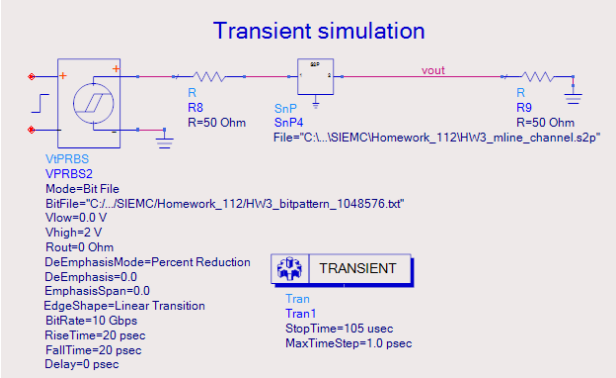
Homework: Time-frequency transform (3) – Sectional convolution

As discussed in the class note, if the bit rate is 10 Gbps, then 2^{20} bits correspond to 105 us. Transient simulation of such a long duration costs a lot of time. In this homework, we are going implement the sectional convolution technique, and compare the efficiency and accuracy with standard circuit simulation using commercial tools.

The circuit is shown below. The channel is a simple 20-cm microstrip line. The S parameters are simulated first, from DC to 40 GHz. The S2P file is exported and used for transient simulation. This is a common practice, especially when the channel is too complicated to be represented by basic circuit components and needs to be simulated by full-wave EM to obtain its S parameters.





The S2P is provided to you in "HW3_mline_channel.s2p". Your main task is to perform sectional convolution for this channel to obtain the time-domain waveform (vout) at the receiver, and compare with that from commercial tools. Note that because the Tx and Rx have source/load impedance equal to the reference impedance of the S2P, i.e., 50 Ω, for this problem we only need to use S_{21}, but not S_{11} nor S_{22}. The input is a random bit pattern with length 2^{20} = 1048576. The bit file is provided to you in "HW3_bitpattern_1048576.txt". The received waveform (vout) simulated by ADS is provided to you in "HW3_Rx_waveform_ADS.txt".

Parameters: Bit rate = 10 Gbps. Risetime = falltime = 20 ps (this is the 0~100% risetime of the trapezoidal NRZ waveform). Incident wave Vpp = 1 V. (Note that in the above circuit, due to voltage division, the PRBS source has Vhigh = 2 V. This results in incident wave of 1 V.) All parameters related to sectional convolution are at your discretion.

Step 1: identify the duration (T_2) of the impulse response of the transfer function S_{21}

(Note: T_2 is theoretically infinite. In addition, spectrum truncation will also cause slow-decaying ripples in the impulse response. In practice, we need to choose a point to truncate the impulse response. The longer T_2, the higher accuracy. One practical choice is to truncate at the point where the tail of the response is 80 dB smaller than the peak value of the impulse response. This ensures

that if the peak is, say 1 V, then the truncated tail will be less than 0.1 mV, which may be small enough to ignore.)

Step 2: read input bit-pattern into Python

Step 3: set parameters for sectional convolution

Step 4: sectional convolution

Step 4-1: construct input waveform $x_k(t)$ of the k-th section

Step 4-2: perform convolution of the k-th section via FFT and IFFT

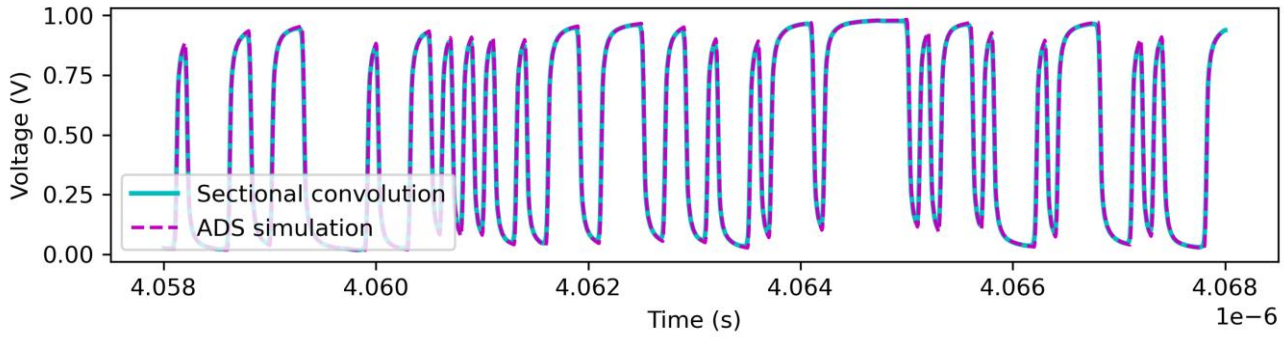
Step 5: load ADS's simulation

Step 6: plot and compare results

In your report, provide the following items:

- List all the parameters that you use (all kinds of parameters in this homework).
- Four snapshots of the output waveform, at different timing, each with width 10 ns, in comparison with ADS's result (see example below). The agreement should be very good.
- Time consumption for sectional convolution on your computer
- Python source code

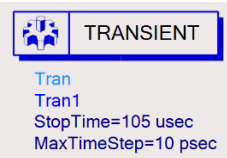
Appendix 1: example answer



Time consumption (on a PC with RAM 120 GB and Intel i9 CPU):

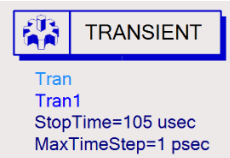
Sectional convolution: 18 sec.

ADS simulation: 441 sec if MaxTimeStep = 10 ps; 3212 sec if MaxTimeStep = 1 ps.




```
TRAN Tran1[1] <Course_material_lib:1130304_Sectional_convolution:schematic>   time=(0 s->105 us)

Resource usage:
  Heap Count = 168 Kitems  Size = 48 MB  Overhead = 2 MB   Total = 50 MB
  Total stopwatch time     =   440.64 seconds.
```

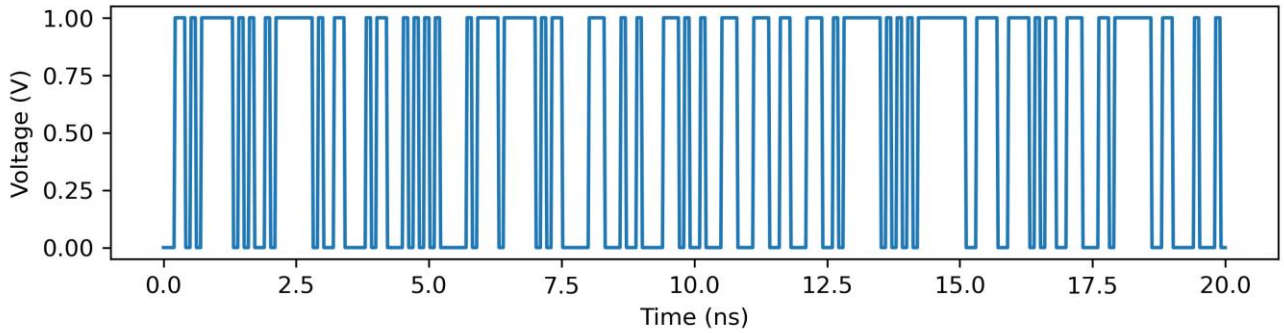



```
TRAN Tran1[1] <Course_material_lib:1130304_Sectional_convolution:schematic>   time=(0 s->105 us)

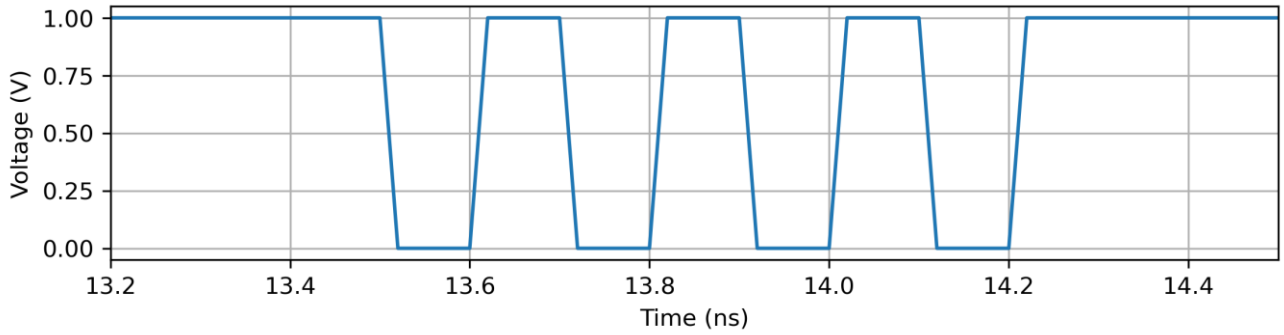
Resource usage:
  Heap Count = 174 Kitems  Size = 48 MB  Overhead = 2 MB   Total = 50 MB
  Total stopwatch time     =  3211.93 seconds.
```

Appendix 2: about the input waveform

Constructing the correct input waveform is crucial for obtaining good agreement with ADS simulation. You may need to plot the input waveform that you constructed to check if it is correct. An example section of the input waveform is shown below:



This homework requires that risetime = 20 ps. Enlarging the above waveform around 14 ns, we have



The above waveform is in exact agreement with the ADS's input waveform by the VtPRBS source.

Appendix 3: Kaiser window

For this homework, since we already construct the exact input waveform with risetime consideration, and since the risetime is finite which provides a natural lowpass effect on the input spectrum, use of window functions with strong attenuation at the boundary is not needed. That means when doing IDFT, the parameter α of the Kaiser window need not be large. You can try different values to see its effect.

Finally, you are encouraged to run ADS simulation yourself to get a feeling of how long a 105-us transient simulation would take. If you don't have access to ADS, contact the instructor via eeclass.